

Policies

- Due 11:59 PM PST, January 13th on Gradescope.
- You are free to collaborate on all of the problems, subject to the collaboration policy stated in the syllabus.
- If you have trouble with this homework, it may be an indication that you should drop the class.
- In this course, we will be using Google Colab for code submissions. You will need a Google account.
- You are allowed to use up to a total of 48 late hours throughout the term. Late hours must be used in units of whole hours. Specify the total number of hours you have ever used when turning in the assignment.
- Students are expected to complete homework assignments based on their understanding of the course material. Student can use LLMs as a resource (e.g., helping with debugging, or grammar checking), but the assignments (including code) should be principally authored by the student.

Submission Instructions

- Submit your report as a single .pdf file to Gradescope (entry code WNB8V4), under "Problem Set 1".
- In the report, **include any images generated by your code** along with your answers to the questions.
- Submit your code by **sharing a link in your report** to your Google Colab notebook for each problem (see naming instructions below). Make sure to set sharing permissions to at least "Anyone with the link can view". **Links that can not be run by TAs will not be counted as turned in.** Check your links in an incognito window before submitting to be sure.
- For instructions specifically pertaining to the Gradescope submission process, see https://www.gradescope.com/get_started#student-submission.

Google Colab Instructions

For each notebook, you need to save a copy to your drive.

1. Upload all the files found in the 'code' folder from the set1.zip to your Google drive (put them all in one folder).
2. Edit the .ipynb file names to "lastname_firstname_originaltitle", e.g. "yue_yisong_3_notebook_part1.ipynb"

Folder with the scripts in google drive

<https://drive.google.com/drive/folders/1HOI59SZ7dmv61IZczIE2cvJjAXhSN8nT?usp=sharing>

1 Basics [16 Points]

Relevant materials: lecture 1

Answer each of the following problems with 1-2 short sentences.

Problem A [2 points]: What is a hypothesis set?

Solution A: *The hypothesis set is the space of candidate functions a learning algorithm can choose from to map inputs X to outputs Y .*

Problem B [2 points]: What is the hypothesis set of a linear model?

Solution B: *The hypothesis set is the set of all hyperplanes in the input space.*

A linear model is defined as

$$h_{w,b}(x) = w^\top x + b,$$

where $w \in \mathbb{R}^d$ and $b \in \mathbb{R}$.

The corresponding hypothesis set is

$$\mathcal{H} = \{h_{w,b} : h_{w,b}(x) = w^\top x + b, w \in \mathbb{R}^d, b \in \mathbb{R}\}.$$

Problem C [2 points]: What is overfitting?

Solution C: *It occurs when the model learns the training data too well, including the noise, and as a result, it fails to generalize to the new unseen data.*

Problem D [2 points]: What are two ways to prevent overfitting?

Solution D: *Overfitting can be prevented by regularization (e.g. L1 or L2 penalization) and by controlling model complexity (e.g., early stopping or using simpler models).*

Problem E [2 points]: What are training data and test data, and how are they used differently? Why should you never change your model based on information from test data?

Solution E: *The train data is used to learn the model parameters, where we can observe both X features and Y labels. Test data is used only to evaluate generalization, i.e., to assess how well our model performs on unseen data. If we allow using the test data to modify the model, this will lead us to data leakage, so the test data will no longer be unseen; therefore, the reported performance becomes biased. This breaks the assumption that the training data should be independent of the training procedure.*

Problem F [2 points]: What are the two assumptions we make about how our dataset is sampled?

Solution F: *We assume that the data points are sampled i.i.d. from an underlying distribution, and both training and test datasets are drawn from the same distribution.*

Problem G [2 points]: Consider the machine learning problem of deciding whether or not an email is spam. What could X , the input space, be? What could Y , the output space, be?

Solution G: *We can convert all words into binary features, yielding a bag-of-words representation. We can also include the number of external links in the text and a binary vector indicating the presence of suspicious words. The output Y could be 1 in case the email is spam and 0 otherwise.*

Problem H [2 points]: What is the k -fold cross-validation procedure?

Solution H: *k -fold cross-validation is used to reduce overfitting when data are limited. k -fold cross-validation splits the data into k folds, trains the model k times, each time holding out one fold for validation, and averages the validation errors to estimate generalization performance. k -fold cross-validation is done without using the test set.*

2 Bias-Variance Tradeoff [34 Points]

Relevant materials: lecture 1

Problem A [5 points]: Derive the bias-variance decomposition for the squared error loss function. That is, show that for a model f_S trained on a dataset S to predict a target $y(x)$ for each x ,

$$\mathbb{E}_S [E_{\text{out}}(f_S)] = \mathbb{E}_x [\text{Bias}(x) + \text{Var}(x)]$$

given the following definitions:

$$\begin{aligned} F(x) &= \mathbb{E}_S [f_S(x)] \\ E_{\text{out}}(f_S) &= \mathbb{E}_x [(f_S(x) - y(x))^2] \\ \text{Bias}(x) &= (F(x) - y(x))^2 \\ \text{Var}(x) &= \mathbb{E}_S [(f_S(x) - F(x))^2] \end{aligned}$$

Solution A: We start from the definition of the expected out-of-sample error:

$$E_{\text{out}}(f_S) = \mathbb{E}_x [(f_S(x) - y(x))^2].$$

Taking expectation over datasets S ,

$$\mathbb{E}_S [E_{\text{out}}(f_S)] = \mathbb{E}_S [\mathbb{E}_x ((f_S(x) - y(x))^2)] = \mathbb{E}_x [\mathbb{E}_S ((f_S(x) - y(x))^2)].$$

Add and subtract $F(x) = \mathbb{E}_S [f_S(x)]$:

$$f_S(x) - y(x) = (f_S(x) - F(x)) + (F(x) - y(x)).$$

Squaring,

$$(f_S(x) - y(x))^2 = (f_S(x) - F(x))^2 + (F(x) - y(x))^2 + 2(f_S(x) - F(x))(F(x) - y(x)).$$

Taking expectation over S ,

$$\mathbb{E}_S [(f_S(x) - y(x))^2] = \mathbb{E}_S [(f_S(x) - F(x))^2] + (F(x) - y(x))^2 + 2(F(x) - y(x))\mathbb{E}_S [f_S(x) - F(x)].$$

Since

$$\mathbb{E}_S [f_S(x) - F(x)] = \mathbb{E}_S [f_S(x)] - F(x) = F(x) - F(x) = 0,$$

the cross term vanishes.

Thus,

$$\mathbb{E}_S [(f_S(x) - y(x))^2] = \underbrace{\mathbb{E}_S [(f_S(x) - F(x))^2]}_{\text{Var}(x)} + \underbrace{(F(x) - y(x))^2}_{\text{Bias}(x)}.$$

Finally,

$$\mathbb{E}_S [E_{\text{out}}(f_S)] = \mathbb{E}_x [\text{Bias}(x) + \text{Var}(x)].$$

In the following problems you will explore the bias-variance tradeoff by producing learning curves for polynomial regression models.

A *learning curve* for a model is a plot showing both the training error and the cross-validation error as a function of the number of points in the training set. These plots provide valuable information regarding the bias and variance of a model and can help determine whether a model is over- or under-fitting.

Polynomial regression is a type of regression that models the target y as a degree- d polynomial function of the input x . (The modeler chooses d .) You don't need to know how it works for this problem, just know that it produces a polynomial that attempts to fit the data.

Problem B [14 points]: Use the provided `2_notebook.ipynb` Jupyter notebook to enter your code for this question. This notebook contains examples of using NumPy's `polyfit` and `polyval` methods, and scikit-learn's `KFold` method; you may find it helpful to read through and run this example code prior to continuing with this problem. Additionally, you may find it helpful to look at the documentation for scikit-learn's `learning_curve` method for some guidance.

The dataset `bv_data.csv` is provided and has a header denoting which columns correspond to which values. Using this dataset, plot learning curves for 1st-, 2nd-, 6th-, and 12th-degree polynomial regression (4 separate plots) by following these steps for each degree $d \in \{1, 2, 6, 12\}$:

1. For each $N \in \{20, 25, 30, 35, \dots, 100\}$:
 - i. Perform 5-fold cross-validation on the first N points in the dataset (setting aside the other points), computing the both the training and validation error for each fold.
 - Use the mean squared error loss as the error function.
 - Use NumPy's `polyfit` method to perform the degree- d polynomial regression and NumPy's `polyval` method to help compute the errors. (See the example code and [NumPy documentation](#) for details.)
 - When partitioning your data into folds, although in practice you should randomize your partitions, for the purposes of this set, simply divide the data into K contiguous blocks.
 - ii. Compute the average of the training and validation errors from the 5 folds.
2. Create a learning curve by plotting both the average training and validation error as functions of N .
Hint: Have same y-axis scale for all degrees d .

Link for 2_notebook.ipynb

<https://drive.google.com/file/d/19hOLnzThQWtFruV31ARokdAXfgW0uzav/view?usp=sharing>

Solution B:

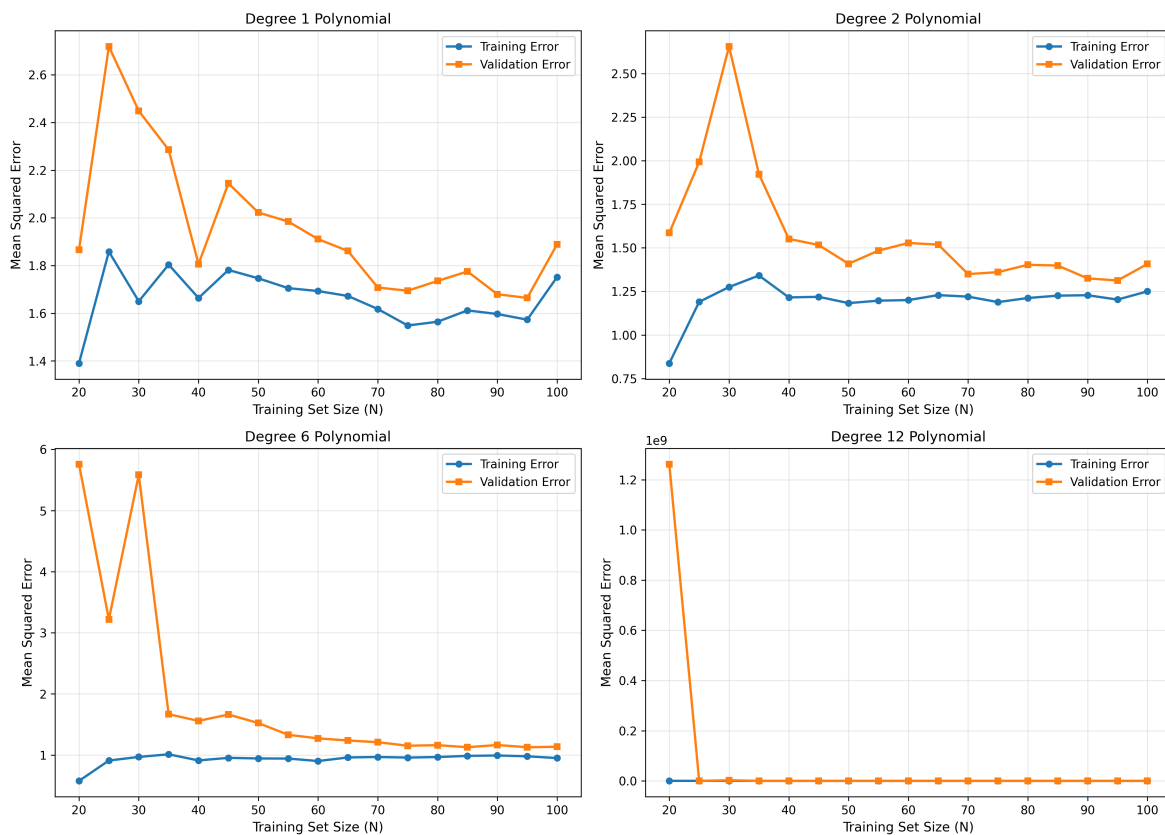


Figure 1: Experiment 3: Training and validation learning curves.

Problem C [3 points]: Based on the learning curves, which polynomial regression model (i.e. which degree polynomial) has the highest bias? How can you tell?

Solution C: Based on the learning curves degree 1 has the highest bias. Both errors for training and validation remain high. Even at $N=100$ Training Error: 1.752 Validation Error: 1.889. These are the highest errors among all models. Even as N increases from 20 to 100 the errors do not decrease much, this suggests that more data does not help.

Problem D [3 points]: Which model has the highest variance? How can you tell?

Solution D: *Polynomial of degree 12 has the highest variance. There is extreme overfitting at small N. At N=20 Training Error: 0.156, Validation Error: 1.2 billion, basically the polynomial perfectly memorizes the training data but produces inaccurate predictions on the validation data. Also, the validation error decreases sharply as N increases, this means the model needs a lot of data to stabilize which is a sign of high variance.*

Problem E [3 points]: What does the learning curve of the quadratic model tell you about how much the model will improve if we had additional training points?

Solution E: *For the quadratic model, adding more training points would likely provide just minimal improvements. The gap between the two curves is small and relatively constant as N increases, the model is not overfitting or underfitting just reached its capacity. This means that the model has learned everything it can from the data given its complexity(approx when N=50), adding more training data will not reduce the validation/training error.*

Problem F [3 points]: Why is training error generally lower than validation error?

Solution F: *This happens because the model is optimized on the training data, basically the model "sees" the training points when learning and the validation data is never seen during this process. So any noise in the training set that the model learned will not help on validation data.*

Problem G [3 points]: Based on the learning curves, which model would you expect to perform best on some unseen data drawn from the same distribution as the training data, and why?

Solution G: *I think degree 6 polynomial will perform best on unseen data. If we see the validation error at N=100 for degree 6 polynomial it has the lowest value (1.135) compared to the other polynomial. Also it achieves the best bias-variance tradeoff. Unlike degree 1 (high bias), degree 6 is flexible enough to capture the underlying pattern. Unlike degree 12 (high variance), degree 6 does not overfit. Also when checking the validation error decreases with the sample size and its approaching to the training error, which means is stable and improving steadily.*

3 Stochastic Gradient Descent [36 Points]

Relevant materials: lecture 2

Stochastic gradient descent (SGD) is an important optimization method in machine learning, used everywhere from logistic regression to training neural networks. In this problem, you will be asked to first implement SGD for linear regression using the squared loss function. Then, you will analyze how several parameters affect the learning process.

Linear regression learns a model of the form:

$$f(x_1, x_2, \dots, x_d) = \left(\sum_{i=1}^d w_i x_i \right) + b$$

Problem A [2 points]: We can make our algebra and coding simpler by writing $f(x_1, x_2, \dots, x_d) = \mathbf{w}^T \mathbf{x}$ for vectors $\mathbf{w}$ and $\mathbf{x}$. But at first glance, this formulation seems to be missing the bias term b from the equation above. How should we define $\mathbf{x}$ and $\mathbf{w}$ such that the model includes the bias term?

Hint: Include an additional element in $\mathbf{w}$ and $\mathbf{x}$.

Solution A:

$$\mathbf{w} = \begin{bmatrix} b \\ w_1 \\ w_2 \\ \vdots \\ w_d \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} 1 \\ x_1 \\ x_2 \\ \vdots \\ x_d \end{bmatrix}$$

Linear regression learns a model by minimizing the squared loss function L , which is the sum across all training data $\{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_N, y_N)\}$ of the squared difference between actual and predicted output values:

$$L(f) = \sum_{i=1}^N (y_i - \mathbf{w}^T \mathbf{x}_i)^2$$

Problem B [2 points]: SGD uses the gradient of the loss function to make incremental adjustments to the weight vector $\mathbf{w}$. Derive the gradient of the squared loss function with respect to $\mathbf{w}$ for linear regression.

Solution B:

$$L(\mathbf{w}) = \sum_{i=1}^N (y_i - \mathbf{w}^T \mathbf{x}_i)^2$$

The gradient of the loss with respect to $\mathbf{w}$ is

$$\nabla_{\mathbf{w}} L(\mathbf{w}) = -2 \sum_{i=1}^N (y_i - \mathbf{w}^T \mathbf{x}_i) \mathbf{x}_i = 2 \sum_{i=1}^N (\mathbf{w}^T \mathbf{x}_i - y_i) \mathbf{x}_i.$$

For stochastic gradient descent (single example i), the gradient is

$$\nabla_{\mathbf{w}} \ell_i(\mathbf{w}) = -2 (y_i - \mathbf{w}^\top \mathbf{x}_i) \mathbf{x}_i.$$

The following few problems ask you to work with the first of two provided Jupyter notebooks for this problem, `3_notebook_part1.ipynb`, which includes tools for gradient descent visualization. This notebook utilizes the files `sgd_helper.py` and `multiopt.mp4`, but you should not need to modify either of these files.

For your implementation of problems C-E, **do not** consider the bias term.

Problem C [8 points]: Implement the `loss`, `gradient`, and `SGD` functions, defined in the notebook, to perform SGD, using the guidelines below:

- Use a squared loss function.
- Terminate the SGD process after a specified number of epochs, where each epoch performs one SGD iteration for each point in the dataset.
- It is recommended, but not required, that you shuffle the order of the points before each epoch such that you go through the points in a random order. You can use `numpy.random.permutation`.
- Measure the loss after each epoch. Your `SGD` function should output a vector with the loss after each epoch, and a matrix of the weights after each epoch (one row per epoch). Note that the weights from all epochs are stored in order to run subsequent visualization code to illustrate SGD.

The link for `3_notebook_part1.ipynb`

https://drive.google.com/file/d/1FrlbiMQrbU_jlo-P5HtFpJGtPds1z5mD/view?usp=sharing

Solution C: See code.

Problem D [2 points]: Run the visualization code in the notebook corresponding to problem D. How does the convergence behavior of SGD change as the starting point varies? How does this differ between datasets 1 and 2? Please answer in 2-3 sentences.

Solution D: Changing the starting point changes the path and speed of SGD, but not the final minimum (for convex losses). With different initial points we have different gradient directions initially, SGD follows different trajectories down the loss surface, but all these paths converge to the minimum, but with different routes. When we use different datasets, this will induce different curvature of the loss surface, which affects how

SGD converges, even with the same model.

Problem E [6 points]: Run the visualization code in the notebook corresponding to problem E. One of the cells—titled “Plotting SGD Convergence”—must be filled in as follows. Perform SGD on dataset 1 for each of the learning rates $\eta \in \{1e-6, 5e-6, 1e-5, 3e-5, 1e-4\}$. On a single plot, show the training error vs. number of epochs trained for each of these values of η . What happens as η changes?

Solution E: As η increases then SGD converges faster, the training error decreases faster per epoch.

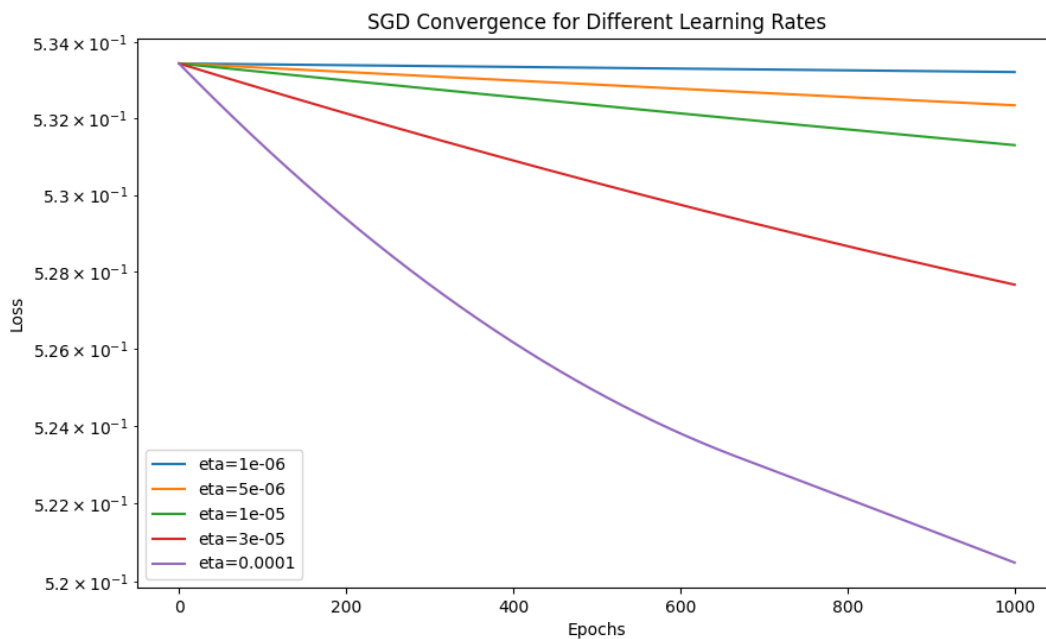


Figure 2: Experiment 3: Training and validation learning curves.

The following problems consider SGD with the larger, higher-dimensional dataset, `sgd_data.csv`. The file has a header denoting which columns correspond to which values. For these problems, use the Jupyter notebook `3_notebook_part2.ipynb`.

For your implementation of problems F-H, **do** consider the bias term using your answer to problem A.

The link for `3_notebook_part2.ipynb`

<https://drive.google.com/file/d/1XBskjMr6eNtnWIWJqZG-kh5UN-ZvPCxi/view?usp=sharing>

Problem F [6 points]: Use your SGD code with the given dataset, and report your final weights. Follow the guidelines below for your implementation:

- Use $\eta = e^{-15}$ as the step size.
- Use $\mathbf{w} = [0.001, 0.001, 0.001, 0.001]$ as the initial weight vector and $b = 0.001$ as the initial bias.
- Use at least 800 epochs.
- You should incorporate the bias term in your implementation of SGD and do so in the vector style of problem A.
- Note that for these problems, it is no longer necessary for the SGD function to store the weights after all epochs; you may change your code to only return the final weights.

Solution F:

Problem G [2 points]: Perform SGD as in the previous problem for each learning rate η in

$$\{e^{-10}, e^{-11}, e^{-12}, e^{-13}, e^{-14}, e^{-15}\},$$

and calculate the training error at the beginning of each epoch during training. On a single plot, show training error vs. number of epochs trained for each of these values of η . Explain what is happening.

Solution G: As the learning rate increases, SGD converges faster, but only up to a point. Very small learning rate leads to slow learning, while sufficiently large learning rate shows rapid convergence.

Problem H [2 points]: The closed form solution for linear regression with least squares is

$$\mathbf{w} = \left(\sum_{i=1}^N \mathbf{x}_i \mathbf{x}_i^T \right)^{-1} \left(\sum_{i=1}^N \mathbf{x}_i y_i \right).$$

Compute this analytical solution. Does the result match up with what you got from SGD?

Solution H: Using the closed-form least-squares solution for linear regression, we obtain

$$\hat{\mathbf{w}}_{LS} = (X^T X)^{-1} X^T \mathbf{y} = \begin{bmatrix} -0.31644 \\ -5.99157 \\ 4.01510 \\ -11.93326 \\ 8.99061 \end{bmatrix}.$$

Running stochastic gradient descent with step size $\eta = e^{-15}$, initial weights $\mathbf{w} = [0.001, 0.001, 0.001, 0.001]$,

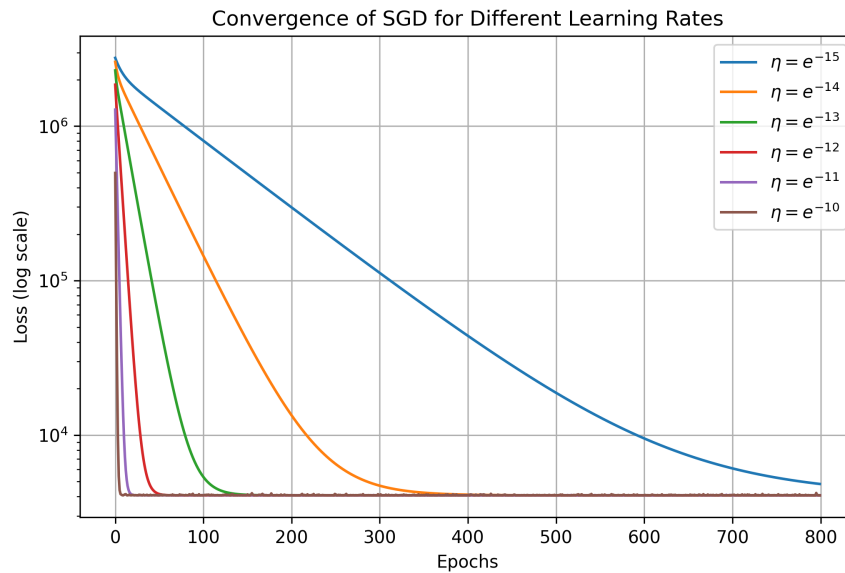


Figure 3: Experiment 3: Training and validation learning curves.

initial bias $b = 0.001$, and 800 epochs, the final SGD iterate is

$$\hat{\mathbf{w}}_{\text{SGD}} = \begin{bmatrix} -0.31611 \\ -5.99377 \\ 4.01377 \\ -11.93728 \\ 8.99395 \end{bmatrix}.$$

The two solutions are extremely close, differing only by small numerical error due to the stochastic nature of SGD and the finite learning rate. This confirms that SGD converges to the same solution as the analytical least-squares estimator.

Answer the remaining questions in 1-2 short sentences.

Problem I [2 points]: Is there any reason to use SGD when a closed form solution exists?

Solution I: computing $(X * X)^{-1}$ can be unstable when features are correlated, SGD avoids explicit matrix inversion. Closed form solution may become infeasible for large datasets because it requires storing the full $(X * X)$, while SGD processes data incrementally.

Problem J [2 points]: Based on the SGD convergence plots that you generated earlier, describe a stopping

condition that is more sophisticated than a pre-defined number of epochs.

Solution J: A more sophisticated stopping condition is to stop training once the loss has converged. For example, stop at epoch t if

$$\frac{|L_t - L_{t-1}|}{L_{t-1}} < \epsilon$$

for several consecutive epochs, indicating that further training yields negligible improvement.

Problem K [2 points]: How does the convergence behavior of the weight vector differ between the perceptron and SGD algorithms?

Solution K: The perceptron algorithm only updates the weight vector when it finds a misclassification point, while the SGD updates the weight vector in each iteration over all epochs. PLA weight vector guarantees convergence to a perfect separator if data is linear separable, while the SGD weight vector shows noisy, oscillating convergence due to updates from single samples, it approaches to good solution but potentially not a perfect one.

4 The Perceptron [14 Points]

Relevant materials: lecture 2

The perceptron is a simple linear model used for binary classification. For an input vector $\mathbf{x} \in \mathbb{R}^d$, weights $\mathbf{w} \in \mathbb{R}^d$, and bias $b \in \mathbb{R}$, a perceptron $f : \mathbb{R}^d \rightarrow \{-1, 1\}$ takes the form

$$f(\mathbf{x}) = \text{sign} \left(\left(\sum_{i=1}^d w_i x_i \right) + b \right)$$

The weights and bias of a perceptron can be thought of as defining a hyperplane that divides $\mathbb{R}^d$ such that each side represents an output class. For example, for a two dimensional dataset, a perceptron could be drawn as a line that separates all points of class +1 from all points of class -1.

The PLA (or the Perceptron Learning Algorithm) is a simple method of training a perceptron. First, an initial guess is made for the weight vector $\mathbf{w}$. Then, one misclassified point is chosen arbitrarily and the $\mathbf{w}$ vector is updated by

$$\begin{aligned} \mathbf{w}_{t+1} &= \mathbf{w}_t + y(t)\mathbf{x}(t) \\ b_{t+1} &= b_t + y(t), \end{aligned}$$

where $\mathbf{x}(t)$ and $y(t)$ correspond to the misclassified point selected at the t^{th} iteration. This process continues until all points are classified correctly.

The following few problems ask you to work with the provided Jupyter notebook for this problem, titled `4_notebook.ipynb`. This notebook utilizes the file `perceptron_helper.py`, but you should not need to modify this file.

Problem A [8 points]: Implement the `update_perceptron` and `run_perceptron` methods in the notebook, and perform the perceptron algorithm with initial weights $w_1 = 0, w_2 = 1, b = 0$.

Give your solution in the form a table showing the weights and bias at each timestep and the misclassified point $([x_1, x_2], y)$ that is chosen for the next iteration's update. You can iterate through the three points in any order. Your code should output the values in the table below; cross-check your answer with the table to confirm that your perceptron code is operating correctly.

t	b	w_1	w_2	x_1	x_2	y
0	0	0	1	1	-2	+1
1	1	1	-1	0	3	+1
2	2	1	2	1	-2	+1
3	3	2	0			

Include in your report both: the table that your code outputs, as well as the plots showing the perceptron's classifier at each step (see notebook for more detail).

The link for 4_notebook.ipynb

<https://drive.google.com/file/d/1Lpb1IsH333nYJK284hk2EHmthQQuAz0I/view?usp=sharing>

Solution A:

t	b	w1	w2	x1	x2	y
0	0.0	0.0	1.0	1.0	-2.0	1
1	1.0	1.0	-1.0	0.0	3.0	1
2	2.0	1.0	2.0	1.0	-2.0	1
3	3.0	2.0	0.0			

final w = [2. 0.], final b = 3.0

Figure 4: Perceptron results

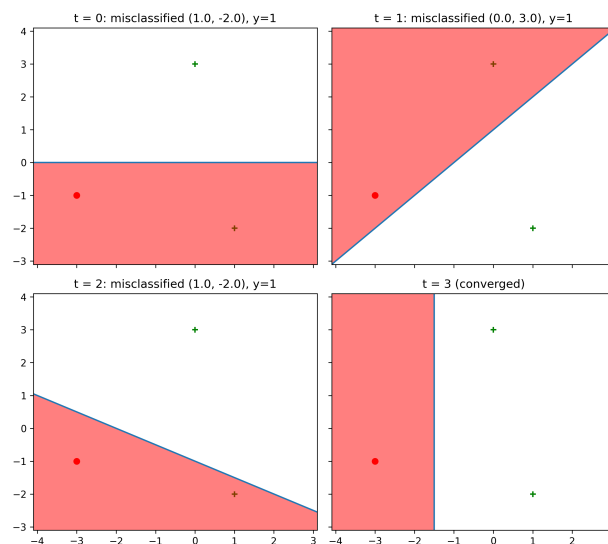


Figure 5: Perceptron results

Problem B [4 points]: A dataset $S = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_N, y_N)\} \subset \mathbb{R}^d \times \mathbb{R}$ is *linearly separable* if there exists a perceptron that correctly classifies all data points in the set. In other words, there exists a hyperplane that separates positive data points and negative data points.

In a 2D dataset, how many data points are in the smallest dataset that is not linearly separable, such that no three points are collinear? How about for a 3D dataset such that no four points are coplanar? Please limit your solution to a few lines - you should justify but not prove your answer.

Finally, how does this generalize for an N -dimensional set, in which **no** $< N$ -dimensional hyperplane contains a non-linearly-separable subset? For the N -dimensional case, you may state your answer without proof or justification.

Solution B: Any three non-collinear points in $\mathbb{R}^2$ are linearly separable. With four points, one can place them in convex position and assign labels in an alternating pattern around the convex hull, which no line can separate.

Similarly, any four points in general position in $\mathbb{R}^3$ are linearly separable. With five points, however, one can construct a configuration consisting of the vertices of a simplex together with an additional point such that one class lies inside the convex hull of the other, preventing separation by a plane.

In general, in $\mathbb{R}^N$, the smallest dataset that is not linearly separable consists of $N + 2$ points.

Problem C [2 points]: Run the visualization code in the Jupyter notebook section corresponding to question C (report your plots). Assume a dataset is *not* linearly separable. Will the Perceptron Learning Algorithm ever converge? Why or why not?

Solution C: If the dataset is not linearly separable, the perceptron learning algorithm will not converge in general. PLA stops when it finds a (w, b) such that every point is correctly classified:

$$y_i (\mathbf{w}^\top \mathbf{x}_i + b) > 0 \quad \forall i.$$

When the data is not linearly separable, there is no such a hyperplane, therefore there will always be at least one misclassified point.